



Technical Documentation

dm-Verity Autoboot Core Module (dm_verity_autoboot.c)

Team Oriented Project

Group A

Linux Kernel Extension for DM-Verity Authentication, Verification and Setup

Date: **NOVEMBER 2025**

Cooperation with:

**BMW
GROUP**



ROLLS-ROYCE
MOTOR CARS LTD

[BMW Car IT, Ulm](#)

Eva-Helena Zorman

Tobias Kaufmann

Philipp Graf

File Location

linux/drivers/md/dm-verity-autoboot/dm_verity_autoboot.c

1. Architectural Overview and Kernel Integration

The `dm_verity_autoboot.c` module is the orchestrator for the entire verified boot sequence, acting as a kernel `late_initcall`. This ensures that the code executes after all core drivers (like the block device layer) are operational. Its primary goal is to securely translate kernel command line arguments into a functional Device Mapper target (`/dev/dm-0`), which the kernel can then trust for the root filesystem. The entire process is non-blocking to the main kernel thread flow, running inside a delayed work queue to allow device hot-plugging to settle.

2. Module Constants and On-Disk Structures

The module defines the fixed security and structural parameters for the metadata.

Constants:

```
56 #define VERITY_META_SIZE 4096
57 #define VERITY_META_MAGIC 0x56455249 /* "VERI" */
58 #define VERITY_FOOTER_SIGNED_LEN 196 /* bytes covered by PKCS7 */
59 #define VERITY_PKCS7_MAX 2048
60 #define VLOC_MAGIC 0x564C4F43 /* "VLOC" */
```

- **VERITY_META_SIZE (4096)**: Defines the fixed size (4 KiB) of the footer block that is always read from the end of the block device to check for the magic number.
- **VERITY_FOOTER_SIGNED_LEN (196)**: This is the fixed number of bytes of the metadata header that is hashed and signed when using the attached ("VERI") format.

Attached Metadata Structure (VERI Magic):

```
71 struct verity_metadata_ondisk {
72     __le32 magic;
73     __le32 version;
74     __le64 data_blocks;
75     __le64 hash_start_sector;
76     __le32 data_block_size;
77     __le32 hash_block_size;
78     char hash_algorithm[32];
79     u8 root_hash[64];
80     u8 salt[64];
81     __le32 salt_size;
82     __le32 pkcs7_size;
83     u8 pkcs7_blob[2048];
84     u8 reserved[4096 - 2248];
85 } __packed;
```

This structure is a direct memory representation of the last 4 KiB of the disk for the attached format. The `__packed` attribute ensures no compiler padding is introduced, maintaining bit-

for-bit accuracy with the on-disk format. It holds both the metadata header fields and the embedded PKCS#7 signature blob.

Detached Locator Structure (VLOC Magic):

```
92  struct verity_footer_locator {
93      __le32 magic;
94      __le32 version;
95      __le64 meta_off;
96      __le32 meta_len;
97      __le64 sig_off;
98      __le32 sig_len;
99      u8      reserved[4096 - 32];
100 } __packed;
```

This structure defines the layout of the 4 KiB locator when the metadata and signature are stored detached elsewhere on the disk. It only contains the necessary offsets (`_off`) and lengths (`_len`) to locate the actual verification assets.

3. External Helpers and Core I/O

The module defines or utilizes several essential kernel I/O and crypto helpers.

3.1. Hashing and Digest Computation

The SHA-256 logic is encapsulated for repeated use:

```
116 static int sha256_buf(const u8 *buf, size_t len, u8 digest[32])
117 {
118     struct crypto_shash *tfm;
119     struct shash_desc *desc;
120     int ret;
121
122     tfm = crypto_alloc_shash("sha256", 0, 0);
123     if (IS_ERR(tfm))
124         return PTR_ERR(tfm);
125
126     desc = kzalloc(sizeof(*desc) + crypto_shash_descsize(tfm), GFP_KERNEL);
127     if (!desc) {
128         crypto_free_shash(tfm);
129         return -ENOMEM;
130     }
131
132     desc->tfm = tfm;
133     ret = crypto_shash_init(desc);
134     if (ret)
135         goto out;
136     ret = crypto_shash_update(desc, buf, len);
137     if (ret)
138         goto out;
139     ret = crypto_shash_final(desc, digest);
140
141 out:
142     kfree(desc);
143     crypto_free_shash(tfm);
144     return ret;
145 }
146
```

- **sha256_buf()**: A full kernel wrapper that allocates a SHA-256 transformation object (struct crypto_shash) and computes the hash of an arbitrary buffer (buf) of length len. It ensures proper cleanup of cryptographic resources.
- **compute_footer_digest()**: A specialized wrapper for the attached metadata flow:

```

148 static int compute_footer_digest(const struct verity_metadata_ondisk *meta,
149 | | | | u8 digest[32])
150 {
151 |     return sha256_buf((const u8 *)meta, VERITY_FOOTER_SIGNED_LEN, digest);
152 }

```

This function is critical because it hashes precisely the first VERITY_FOOTER_SIGNED_LEN(196) bytes, which is the fixed, signed data region for the attached format.

3.2. Block Device Operations

The module must perform raw reads on the block device.

- **read_region()**: The universal reading utility for detached metadata:

```

157 static int read_region(struct file *bdev_file, u64 off, u32 len, u8 **out)
158 {
159 |     loff_t pos = off;
160 |     ssize_t rd;
161 |     u8 *buf;
162
163 |     if (!len || len > (8U << 20))
164 |         return -EINVAL;
165
166 |     buf = kmalloc(len, GFP_KERNEL);
167 |     if (!buf)
168 |         return -ENOMEM;
169
170 |     rd = kernel_read(bdev_file, buf, len, &pos);
171 |     if (rd != len) {
172 |         kfree(buf);
173 |         return rd < 0 ? (int)rd : -EIO;
174 |     }
175
176 |     *out = buf;
177 |     return 0;
178 }

```

This function handles dynamic memory allocation (kmalloc) for the detached regions and performs the raw read via kernel_read(), which updates the file position (&pos). It strictly checks that the requested number of bytes (len) was actually read, returning -EIO otherwise.

- **resolve_dev_from_diskname()**: Discussed previously, this function is critical for translating the kernel command line string (e.g., "vda") into the device identifier (dev_t), allowing the module to open the device before /dev/ nodes are finalized.

4. Initialization and Main Worker Orchestration

4.1. Initialization (dm_verity_autoboot_init)

This function is registered as a `late_initcall`. It logs the command line and, if `autoboot_device` is set, schedules the main worker:

```
673 |     if (autoboot_device && *autoboot_device) {  
674 |         schedule_delayed_work(&verity_work, 5 * HZ);
```

The use of `schedule_delayed_work()` with a 5-second delay (`5 * HZ`) is a deliberate design choice to ensure device availability. It provides time for device initialization (e.g., reading SCSI/virtio identifiers) to complete before the module attempts to open the file descriptor.

4.2. Main Worker (verity_autoboot_main)

This function is the execution core. It first resolves the device path and opens the block device file:

```
328 |     ret = resolve_dev_from_diskname(autoboot_device, &dev);  
  
/ ... error checking ... /  
  
338 |     bdev_file = bdev_file_open_by_dev(dev, BLK_OPEN_READ, NULL, NULL);  
339 |     if (IS_ERR(bdev_file)) {  
340 |         pr_err("%s: bdev_file_open_by_dev() -> %ld\n",  
341 |             DM_MSG_PREFIX, PTR_ERR(bdev_file));  
342 |         return PTR_ERR(bdev_file);  
343 |     }
```

The absolute critical moment is the read of the final 4 KiB (`VERITY_META_SIZE`) of the device, which is necessary to check the magic number and branch the execution:

```
348 |     loff_t sz = i_size_read(file_inode(bdev_file));  
349 |     loff_t pos = sz - VERITY_META_SIZE;  
350 |     ssize_t got = kernel_read(bdev_file, tail, VERITY_META_SIZE, &pos);
```

5. Detached Footer Flow ("VLOC")

This path is exclusively taken if the magic number found at the end of the block device is confirmed to be `VLOC_MAGIC`.

This flow is significantly more complex than a conventional embedded footer due to the requirement to locate and validate external, non-contiguous data regions. The complexity stems from the need to parse the locator structure, which contains critical offsets and lengths pointing to both the detached dm-verity metadata header and the PKCS#7 cryptographic signature. This requires multiple I/O operations across disparate sections of the block device. Furthermore, a crucial security prerequisite is the rigorous validation of all disk offsets and lengths against the physical bounds of the block device itself to prevent pointer overflow attacks and ensure data integrity. Only after these multiple data structures are securely read

and validated can the final, CPU-intensive PKCS#7 signature verification be executed against the known public key, thereby establishing the Dynamic Root of Trust for the root filesystem.

5.1.Extensive Bounds and Overflow Validation

This is the most security-sensitive section of the code, dedicated entirely to preventing Time-of-Check to Time-of-Use (TOCTOU) and buffer overflow attacks by strictly validating the pointers (`_off` and `_len`) from the disk locator:

```
467     /* Validate meta region */
468     if (le64_to_cpu(loc.meta_off) >= disk_size) {
469         pr_emerg("%s: VALIDATION FAILED: meta_off %llu beyond disk size %lld\n",
470             DM_MSG_PREFIX,
471             (unsigned long long)le64_to_cpu(loc.meta_off),
472             disk_size);
473         fput(bdev_file);
474
475         return -EINVAL;
476     }
477     if (le32_to_cpu(loc.meta_len) == 0 ||
478         le32_to_cpu(loc.meta_len) > (8U << 20)) { // 8MB sanity limit
479         pr_emerg("%s: VALIDATION FAILED: invalid meta_len %u\n",
480             DM_MSG_PREFIX, le32_to_cpu(loc.meta_len));
481         fput(bdev_file);
482         return -EINVAL;
483     }
484
485     meta_end = le64_to_cpu(loc.meta_off) + le32_to_cpu(loc.meta_len);
486     if (meta_end < le64_to_cpu(loc.meta_off)) {
487
488         pr_emerg("%s: VALIDATION FAILED: meta region overflow (meta_off + meta_len wraps around)\n",
489             DM_MSG_PREFIX);
490         fput(bdev_file);
491         return -EINVAL;
492     }
493     if (meta_end > disk_size) {
494         pr_emerg("%s: VALIDATION FAILED: meta region [%llu, %llu] exceeds disk size %lld\n",
495             DM_MSG_PREFIX,
496             (unsigned long long)le64_to_cpu(loc.meta_off),
497             (unsigned long long)meta_end,
498             disk_size);
499         fput(bdev_file);
500         return -EINVAL;
501     }
```

After the code above, identical validation for sig region follows. These checks are absolutely vital. By ensuring that the locator pointers are within the bounds of the disk, the module guarantees that the subsequent calls to `read_region()` will not attempt to read past the end of the device or use corrupted, overflowed addresses.

5.2. Region Read and Verification

If validation passes, the two regions are read into separate memory buffers:

```
540     ret = read_region(bdev_file, le64_to_cpu(loc.meta_off),
541         le32_to_cpu(loc.meta_len), &meta_buf);
542     if (ret) {
543         pr_err("%s: reading metadata region: %d\n",
544             DM_MSG_PREFIX, ret);
545         fput(bdev_file);
546         return ret;
547     }
```

(read `sig_buf` follows the code above) The detached verification is then executed:

```
566 | | | | | ret = verify_signature_pkcs7_detached(meta_buf,  
567 | | | | | le32_to_cpu(loc.meta_len),  
568 | | | | | sig_buf,  
569 | | | | | le32_to_cpu(loc.sig_len));
```

5.3. Finalization

Upon successful verification, both buffers are confirmed to be authentic. The device file is closed, and the mapping is created using the now-verified metadata header found at the start of `meta_buf`. The dynamically allocated buffers are freed upon completion:

```
610 | | | | | kfree(sig_buf);  
611 | | | | | kfree(meta_buf);  
612 | | | | | return 0;
```

The entire module is a textbook example of early-boot security, rigidly enforcing bounds checks, cryptographic verification against a trusted keyring, and disciplined management of kernel I/O resources before yielding control to the dm-verity kernel driver.

End of Document